Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

[your title here]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: Santiago Cifuentes
Codirector: Santiago Figueira
Buenos Aires, 2026

[YOUR TITLE HERE]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[YOUR TITLE HERE]

[write your abstract here]

Keywords: [write your keywords here]

[acá van los agradecimientos]

CONTENTS

1. Introduction 1
   1.1 Uso y poder de los LLMS 1
   1.2 Pero qué es un LLM? 2
   1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos 2
   1.4 Objetivos y contribuciones 2

2. Theoretical Framework 3
   2.1 Transformers 3
   2.2 Transformer Complexity classes 5
   2.3 Circuit Complexity 6
   2.4 Known results 6

3. General properties 7
   3.1 Transformers Primitives 7
   3.2 Normalization 14
   3.3 Boolean Operations 15

4. Main results 17

5. Conclusions 19

1. INTRODUCTION

1.1 Uso y poder de los LLMS

In 2017, a group of researchers working at Google published a paper introducing a technology that revolutionized society. In *Attention is All You Need*, they presented the transformers, today's core of large language models (LLMs) and generative AI.

This architecture was build intended as text sequence-to-sequence model for machine translation but today has applications in various fields as natural language processing, computer vision or speech recognition.

The first model implementing transformers open for public use was ChatGPT by OpenAI. It made its debut in November 2022 and by December 2024 it reported 300 million global weekly users. Now there are many models competing for this market as Claude, from Anthropic, or Gemini from Google.

These models can be all be grouped under the generative AI tools. This is a technology that is arguably the most rapid technology adopted by society. In the second half of 2025, roughly one in six people worldwide were using generative AI tools. not sure how to cite this https://www.microsoft.com/en-us/research/wp-content/uploads/2026/01/Microsoft-AI-Diffusion-Report-2025-H2.pdf

LLMs are being used not only as chatbots for asking small questions, but are shaping the industry and future. For example, [3] found that more than 17% of the Computer Science's papers published between January 2020 and February 2024 on the *arXiv*, *bioRxiv*, and *Nature* portfolio journals had sentences heavily edited by ChatGPT.

Missing usage in industry

So naturally, the question about their computing power pops up. Are this kind of models really capable of doing the stuff for what they are being used?

- well-suited to parallelism enables transformer models to take full advantage of the power and speed offered by GPUs during both training and inference. This possibility, in turn, unlocked the opportunity to train transformer models on unprecedentedly massive datasets through self-supervised learning.
- chatgpt, claude, gemini,
- every day users
- number of users?
- use in the industry?
- other uses besides the chatbots?

cite this https://www.ib model

1.2 Pero qué es un LLM?

1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos

1.4 Objetivos y contribuciones

2. THEORETICAL FRAMEWORK

2.1 Transformers

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ. It has only one tape from where it reads the input and in an autoregressive way writes in it more tokens. There are two distinguished tokens called stopping symbols. Once the transformer writes one of those it halts. We say that a transformer accepts a word if when it halts the stopping symbol printed is the accepting one (q_{accept}). Correspondingly, a transformer rejects a word if the stopping symbol printed is the rejecting one (q_{reject}).

In each step the transformer maps the content of the tape to a probability distribution over the alphabet. Depending on this distribution the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called **chain of thought**.

The process of generating the next token consists of two separated steps. First a probability distribution over the alphabet for the token is computed by a process called distribution generation. After that, it is necessary to decide the specific token by a process called token selection. These two algorithms together is what defines the transformer. During this work we will analyze and compare two different options for token selection.

For a more realistic analysis of the transformers, we don't allow an infinite tape. The transformers studied in this work, for each input size n, will have a tape of size $\text{budget}(n) + n$. It's important to remark that, since in every step of the transformer a symbol is written, the maximum number of steps a transformer can do for an input of length n is $\text{budget}(n)$.

The process of distribution generation is dependent on many parameters. These parameters, in the real models, are the ones calculated during the training. We refer to them as θ. During this work we assume that the parameters force the transformer to halt exactly when the budget runs out. We consider the parameters as part of the definition of a transformer.

Write an appen
about this decis
We can always
an extra step or
artifact forcing

add somewhere
def of softmax

We note $\text{TF}(\alpha)$ to the token outputted by one step of the transformer TF when the tape content is $\alpha \in \Sigma^*$. The autoregressive token generation is defined recursively. The token outputted after i steps is $\text{TF}^i(\alpha) := \text{TF}^{i-1}(\alpha\text{TF}(\alpha))$ with base case $\text{TF}^1(\alpha) := \text{TF}(\alpha)$. In other words, the output of the ith step is $\sigma_i = \text{TF}^i(\alpha) = \text{TF}(\alpha\sigma_1 \dots \sigma_{i-1})$ where $\sigma_j \in \Sigma$ for all j.

We note $\text{TF}^*(\alpha)$ for the token written by the transformer when it halts, i.e. at the end of the computation, when the input is α. As was said before, the transformer only halts when it prints a stopping symbol (q_{accept} or q_{reject}). Therefore, $\text{TF}^*(\alpha) \in \{q_{accept}, q_{reject}\}$.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ. There are different algorithms for this process in the literature. All the transformers considered in this work use the one presented by [2]. This algorithm consists of four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT), and a stack of L identical decoder layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network

(FF). Each part and layer has its own trainable parameters. Then, we can define the distribution generation algorithm by its parameters $\theta = \{\theta_{\mathrm{TE}}, \theta_{\mathrm{PE}}, \{\theta^l_{\mathrm{ATTN}}, \theta^l_{\mathrm{FF}}\}_{l=0}^{L-1}, \theta_{\mathrm{OUTPUT}}\}$. Let's define each component.

Token Embedding: It's a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathrm{TE}} \in \mathbb{R}^{d\times|\Sigma|}$. For all $\sigma \in \Sigma$, we note $\theta_{\mathrm{TE}}(\sigma) \in \mathbb{R}^d$ to the mapping of token σ.

Position Encoding: It's a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathrm{PE}} \in \mathbb{R}^{d\times \mathrm{budget}(n)+n}$. For all position of the tape $1 \leq i \leq \mathrm{budget}(n) + n$ we note $\theta_{\mathrm{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i.

Self Attention Layer: It's a mapping from $(\mathbb{R}^d)^a$ to $(\mathbb{R}^d)^a$. Given parameters $\theta_{\mathrm{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d}$, the layer it's defined by the algorithm 1. This layer is defined only as a single head attention layer because we can simulate 1 multi-head attention layer with any constantly many heads with multiple single-head attention layers. [10]

Algorithm 1 Causal Self-Attention, ATTN

Require: Parameter $\theta_{\mathrm{ATTN}} = (W_Q, W_K, W_V, W_O)$, Input embedding $h = (h_1, \ldots, h_a) \in \mathbb{R}^d \times \cdots \times \mathbb{R}^d$.
Ensure: Output embedding $h' = (h'_1, \ldots, h'_a) := \mathrm{ATTN}_{\theta_{\mathrm{ATTN}}}(h_1, \ldots, h_a)$.
1: $q_i := W_Q h_i,\ k_i := W_K h_i,\ v_i := W_V h_i,\ \forall i \in [1, \ldots, a]$
2: $s_i := \mathrm{softmax}(\langle q_i, k_1\rangle, \ldots, \langle q_i, k_i\rangle)\|(0, \ldots, 0)$
3: $h'_i := W_O \sum_{j=1}^{a} (s_i)_j v_j$

Feed Forward Network: It's a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\mathrm{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d\times d} \times \mathbb{R}^d \times \mathbb{R}^{d\times d} \times \mathbb{R}^d$, it's defined as $\mathrm{FF}_{\theta_{\mathrm{FF}}}(h) := W_2\ \mathrm{relu}(W_1 h + b1) + b2$, where $\mathrm{relu} : \mathbb{R}^d \to \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

Output Layer: It's a mapping from $\mathbb{R}^d$ to a probability distribution over Σ. Given $\theta_{\mathrm{OUTPUT}} \in \mathbb{R}^{|\Sigma|\times d}$, it's defined as $\mathrm{OUTPUT}_{\theta_{\mathrm{OUTPUT}}}(h) := \mathrm{softmax}(\theta_{\mathrm{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

Algorithm 2 Distribution generator

Require: Transformer parameter $\theta = (\theta_{\mathrm{TE}}, \theta_{\mathrm{PE}}, \{\theta^l_{\mathrm{ATTN}}, \theta^l_{\mathrm{FF}}\}_{l=0}^{L-1}, \theta_{\mathrm{OUTPUT}})$ and tokens of the tape $\alpha \in \Sigma^*$.
Ensure: Output distribution $p_\theta(\cdot|\alpha)$.
1: $h_i^{(0)} \leftarrow \theta_{\mathrm{TE}}(\alpha_i) + \theta_{\mathrm{PE}}(i),\ \forall 1 \leq i \leq |\alpha|$
2: **for** $l = 0, \ldots, L-1$ **do**
3: $\quad (h_1^{(l+0.5)}, \ldots, h_{|\alpha|}^{(l+0.5)}) \leftarrow (h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)}) + \mathrm{ATTN}_{\theta^{(l)}_{\mathrm{ATTN}}}(h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)})$
4: $\quad h_i^{(l+1)} \leftarrow h_i^{(l+0.5)} + \mathrm{FF}_{\theta^{(l)}_{\mathrm{FF}}}(h_i^{(l+0.5)}),\ \forall 1 \leq i \leq |\alpha|$
5: **end for**
6: $p_\theta(\cdot|\alpha) \leftarrow \mathrm{OUTPUT}_{\theta_{\mathrm{OUTPUT}}}(h_{|\alpha|}^{(L)})$

The second component of a transformer is the **token selector**. This algorithm consumes the probability distribution computed by the distribution generator and returns a token.

Lets $p : \Sigma \to [0, 1]$ be the output of the distribution generator. In this work we will study two selectors. The first one always returns the token with the largest probability (argmax over p, where p is the output of the distribution generator). The other, works non-deterministically. It returns the token $\sigma \in \Sigma$ with probability $p(\sigma)$.

I forgot to talk about finite representation. Define good symbols for ∞ y $-\infty$. Properties i use later on that i should explain here:

- $exp(-\infty) = 0$
- $exp(\infty) = \infty$
- $-\infty * \infty = -\infty$
- $\infty * \infty = \infty$
- $x + 0 = x$
- $x - x = 0$
- $x + \infty + \infty + -\infty = 0$ for all x.

Add an image r resenting the tr former architect

Definition 2.1.1 (Deterministic transformer)**.** We call a transformer deterministic when the token selector is the one taking argmax over the distribution. We say that a family of deterministic transformers $\{\mathrm{TF}_n\}_{n\in\mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \mathrm{TF}^*_{|\alpha|}(\alpha) = q_{accept}$$

As mentioned in chapter 1, there is a theoretical gap in the area. está bien dicho theoretical gap? A study of non-determinism in transformers is missing.

When TF is a deterministic transformer, $\mathrm{TF}^i(\alpha)$ is an exact symbol for all $\alpha \in \Sigma^*$ and $i \in \mathbb{N}$. In counterpart, when the token selection process of the transformer samples the distribution, it becomes a random variable. Therefore, we introduce the following definition.

Definition 2.1.2 (Probabilistic transformer)**.** We call probabilistic transformers to the transformers that sample the distribution for selecting the next token. We say that a family of probabilistic transformers $\{\mathrm{TF}_n\}_{n\in\mathbb{N}}$ recognize a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \Pr[\mathrm{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$$

2.2 Transformer Complexity classes

Maybe rethink this title?

As discussed in the first chapter, there are different resources to bound in the transformers to limit their expressive power. In this thesis, we will analyze the effect of bounding the number of steps of chain of thought that the transformers are allowed to make before answering. With that in mind, we define the following complexity classes.

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathrm{CoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathrm{CoT}[T(n), d(n)]$ if and only if there is a deterministic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.2.1 (CoT[$T(n), d(n)$])**.** A language $\mathcal{L}$ is in CoT[$T(n), d(n)$] if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer deterministic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought.

In the same way, we define the classes of languages recognized by probabilistic transformers. Given two functions $T(n)$ and $d(n)$ we define the complexity class RCoT[$T(n), d(n)$]. Informally, a language $\mathcal{L}$ is in RCoT[$T(n), d(n)$] if and only if there is a probabilistic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.2.2 (RCoT[$T(n), d(n)$])**.** A language $\mathcal{L}$ is in RCoT[$T(n), d(n)$] if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer probabilistic transformer, denoted by TF_n with embedding size $d'(n)$, that can output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought with probability higher than 2/3.

2.3 Circuit Complexity

2.4 Known results

3. GENERAL PROPERTIES

In order to better understand $\mathrm{CoT}[T(n), d(n)]$ classes, it was needed to fill a gap in the literature related to the properties of these classes. For that, in this section we introduce a notion of normalization for transformers and prove that $\mathrm{CoT}[T(n), d(n)]$ is closed under boolean operations. As middle step, it was needed to build certain primitives operations which are also explained in this chapter.

3.1 Transformers Primitives

Flagging Tokens and Positions

We will see later on that we will want to be able to identify when a certain token appeared in the tape. Also, we would like to know when a certain position is reached or if a certain vector corresponds to a specific position. This last task is not naturally done because of the extreme parallelism in the model.

For flagging a token we use the token embedding, and in the same way, for flagging a position we use the position encoding. Either way, the strategy is exactly the same.

For each flag that we are looking to add, we will add one to the dimension embedding. Notice that if we are working with a constant number of flags, the $\mathrm{CoT}[T(n), d(n)]$ class does not change. Not sure about this last sentence. Maybe I should expand the idea if I want to say it

The idea for flagging is straight forward. We will have an extra coordinate that will work as the flag. This coordinate will have two possible values to be defined. It will have the on value f_{on} if we are flagging its vector or the off value f_{off}, otherwise.

For example, let's be TF a transformer and θ_{TE} its token embedding function. We will define a transformer TF$'$ with the same behavior, but it will have flagged every vector that came from the token σ_f.

For the sake of clarity, we will add the flag in the last coordinate of the embedding, but this can be done in any position.

Let's define each component of TF$'$ based on the components of TF. First, let's see the token embedding function. It will be the same except that in the new coordinate it will put a V_{on} if the token that is consuming is the one to be flagged and a 0 otherwise.

$$(\theta'_{TE}(\sigma))_j = \begin{cases} (\theta_{TE}(\sigma))_j & \text{if } j \neq d+1 \\ f_{\text{on}} & \text{if } j = d+1 \text{ and } \sigma = \sigma_f \\ f_{\text{off}} & \text{if } j = d+1 \text{ and } \sigma \neq \sigma_f \end{cases}$$

For the position embedding is enough to always put a 0 in the new coordinate. Then is only necessary to extend the matrix of the other layers to the new dimension. This can be done with a new row or column with 0s. The behavior of TF$'$ will be the same as TF.

This mechanism clearly works for adding more dimensions to the embedding. We just add new flags with the same value for f_{on} and f_{off}.

Preserve and ignore coordinates through ATTN and FF

When defining new transformers over existing ones we will want to extend their embedding dimension. We would like to preserve the behavior of the old transformer in the existing coordinates but ignore the new ones. Let's see how to extend the dimensions of the ATTN and FF layers in a way that we accomplish this.

If we want to ignore the ith coordinate through a layer of attention, it's enough to put zeros in the ith column and row of W_Q, W_K, W_V, W_O. This can be done with as many coordinates as desired. Note that because the ith row of W_O is zero, the output vector of the ATTN layer will have a zero in the ith coordinate. This is the point that makes it possible to not just ignore the value of the ith coordinate, but to not affect it. This happens because the result of the attention is added to the original vector.

The same applies in the feed forward layer, putting zeros in the ith column and row of the matrix and vectors of the FF makes it ignore the value of the ith coordinate. Also in the result of the FF will appear a zero in the ith coordinate.

Add vector and constant functions

Let's define a mechanism for adding one specific vector to a flagged one. This technique works even when there are more than one flagged vector.

The flag used for the vectors should have $f_{\text{on}} = 1$ and $f_{\text{off}} = 0$. We will assume the flag is in the last coordinate.

Let's call v the vector that we want to add. We will do it with one FF layer. We will construct a layer such that $FF(h) = \mathbb{I}(h \text{ is flagged}) * v$.

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & \dots & 0 & v_1 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & v_d \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

Notice that if the coordinate for the flag had been the kth instead of the last one, then v would have appeared in the kth column of W_2 instead of the last.

An interesting use of this technique is that allow us to transform any vector in whatever we want, i.e. we can apply constant functions. This is done exploiting the finite representation system. We can saturate the vector to the ∞ and then make it 0 to finally add our objective.

$$h \leftarrow \left(\left(\left(h + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} \right) + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} \right) + \begin{pmatrix} -\infty \\ \vdots \\ -\infty \end{pmatrix} \right) + v$$

Linear transformations

Although it looks that transformers are a model that should be able to perform linear transformations in a native way, it is not the case. Applying a linear transformation to a vector is not direct since in every step of ATTN and FF the result of the layer is added to the original vector instead of replacing it. A sharp reader maybe would think that this can be solved applying the transformation $M - I$ when we want to transform the vector

x to Mx, since $x + (M - I)x = x + Mx - Ix = Mx$. This does not hold in our model since the addition and multiplication in the finite representation of the numbers used is not commutative. For avoiding the representation error, we will extend the embedding dimension to get more memory per vector such that we can store in new coordinates the value of Mx and then replace it in the original ones. After applying this process we will have corrupted all vectors except the one we are interested in.

The strategy is the following. We will apply the transformation to only one vector which was flagged with 1 as f_{on} and -1 as f_{off} in the embedding step. This flag will appear duplicated. Let's assume the flag is in the last two coordinates. For having a clearer notation, we will say that the vector to which we are applying the linear transformation is the fth. Also, we will call M to the matrix of the linear transformation. The linear transformation will be done with two layers of ATTN.

The first layer will compute the result of the transformation and store it in new coordinates added as memory. The second layer will move the result to the right place.

For doing this we will need to extend the embedding dimension with enough coordinates to store the result of the transformation. Therefore, if we are working with a transformer of embedding dimension d and want to do a linear transformation of a vector, we will need to extend the dimension to $2d + 2$. The first d coordinates will be for the original vector, the second d coordinates will be the ones using as memory, and the extra two are for the flags. We will assume that the d coordinates used as memory are set to 0 by the layers previous to this mechanism. This can be achieved maintaining them as 0 since the beginning or transformed to 0 with the primitive described in the previous section.

$$h_f = \begin{pmatrix} x_1 \\ \vdots \\ x_d \\ 0 \\ \vdots \\ 0 \\ 1 \\ 1 \end{pmatrix} \xrightarrow{\text{first ATTN layer}} h_f = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (Mx)_1 \\ \vdots \\ (Mx)_d \\ 1 \\ 1 \end{pmatrix} \xrightarrow{\text{second ATTN layer}} h_f = \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_d \\ 0 \\ \vdots \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

a graphic representation of the technique

Let's construct each of the ATTN layers. In both layers we will need for h_f to only attend to itself. For accomplishing that we will make use of the fact that the flag has value 1 only in h_f and -1 in the other vectors. We asked for the flag by duplicate since we need two ATTN layers and in each layer we are corrupting all vectors except h_f. Therefore, making use of the mechanism explained before to preserve and ignore coordinates through ATTN layers, we will write all the matrix as if we are working with embedding dimension $2d + 1$ but in reality, in the first layer we are using the first copy of the flag (and ignoring/preserving the second) and in the second we are doing it in the other way. Thus, adding the missing row and column of zeros will get us the real values of the matrix.

Let's define the parameters of the ATTN layers for h_f only attending to itself. If we take as query and key matrix as

$$W_Q = W_K = \begin{pmatrix} 0 & \dots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & 0 \\ 0 & \dots & 0 & \infty \end{pmatrix}$$

then the keys and values of the vectors are

$$W_Q h_i = q_i = W_K h_i = k_i = \begin{cases} (0, \dots, 0, \infty * 1) & \text{if } i = f \\ (0, \dots, 0, \infty * -1) & \text{if } i \neq f \end{cases} = \begin{cases} (0, \dots, 0, \infty) & \text{if } i = f \\ (0, \dots, 0, -\infty) & \text{if } i \neq f \end{cases}$$

so the attention values for h_f are 1 for itself and 0 for the rest.

$$\begin{aligned} s_f &= \text{softmax}(& \langle q_f, k_1 \rangle, && \dots, && \langle q_f, k_{f-1} \rangle, && \langle q_f, k_f \rangle) && \|(0, \dots, 0) \\ &= \text{softmax}(& \infty * -\infty, && \dots, && \infty * -\infty, && \infty * \infty) && \|(0, \dots, 0) \\ &= \text{softmax}(& -\infty, && \dots, && -\infty, && \infty) && \|(0, \dots, 0) \\ &= & 0, && \dots, && 0, && 1, && 0, \dots, 0 \end{aligned}$$

$$(s_f)_i = \begin{cases} 1 & \text{if } i = f \\ 0 & \text{if } i \neq f \end{cases}$$

Now that we got for h_f to attend only to itself, let's remember from the definition of the ATTN mechanism (1) that the output of the ATTN layer is $h'_i = W_O \sum_{j=1}^{a} (s_i)_j v_j$. Let's analyze what h'_f looks by now:

$$h'_f = W_O \sum_{j=1}^{a} (s_f)_j v_j = W_O v_f = W_O (W_V h_f)$$

Then, defining W_O and W_V as follows

$$W_V = id^{\mathbb{R}^{(2d+1)\times(2d+1)}} \qquad W_O = \begin{pmatrix} -id^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d}} \\ M & 0^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d}} \\ 0^{\mathbb{R}^{1\times d}} & 0^{\mathbb{R}^{1\times d}} & id^{\mathbb{R}} \end{pmatrix}$$

makes us get

$$h'_f = \begin{pmatrix} -x_1 \\ \vdots \\ -x_d \\ (Mx)_1 \\ \vdots \\ (Mx)_d \\ 1 \end{pmatrix}$$

which added to h_f gives us what we were looking for after the first layer.

For the second ATTN layer, we can use the same mechanism for making h_f only attend to itself. And defining W_V and W_O as follows

$$W_V = id^{\mathbb{R}^{(2d+1)\times(2d+1)}} \qquad W_O = \begin{pmatrix} 0^{\mathbb{R}^{d\times d}} & id^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d}} \\ 0^{\mathbb{R}^{d\times d}} & -id^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d}} \\ 0^{\mathbb{R}^{1\times d}} & 0^{\mathbb{R}^{1\times d}} & id^{\mathbb{R}} \end{pmatrix}$$

we get

$$h'_f = \begin{pmatrix} (Mx)_1 \\ \vdots \\ -(Mx)_d \\ -(Mx)_1 \\ \vdots \\ -(Mx)_d \\ 1 \end{pmatrix}$$

which added to h_f gives us what we were looking for after the second layer.

If we wanted to remove the flag, we could replace the bottom leftmost coordinate of W_O by -1 in both layers.

Max coordinate

We will see later on that we will need to be able to take the maximum of a set of coordinates. The operation we are looking to do is the following:

$$\begin{pmatrix} a \\ b \end{pmatrix} \to \begin{cases} \begin{pmatrix} a \\ 0 \end{pmatrix} & \text{if } a > b \\ \\ \begin{pmatrix} 0 \\ b \end{pmatrix} & \text{if } b > a \end{cases}$$

We will define it for taking the maximum between only two coordinates, but it can be extended to a larger set repeating the operation as a tournament.

This operation can be done with the following primitives:

$$\begin{pmatrix} a \\ b \\ 0 \\ 0 \\ \vdots \end{pmatrix} \xrightarrow{\text{FF layer}} \begin{pmatrix} a \\ b \\ \mathrm{relu}(a-b) \\ \mathrm{relu}(b-a) \\ \vdots \end{pmatrix} \xrightarrow[\text{the coordinates with the relu}]{\text{Multiply twice by } \infty} \begin{pmatrix} a \\ b \\ \infty \times \mathbb{I}(a>b) \\ \infty \times \mathbb{I}(b>a) \\ \vdots \end{pmatrix} \longrightarrow$$

$$\xrightarrow[\substack{\text{coordinates with} \\ \text{the comparison}}]{\text{Divide by } \infty \text{ the}} \begin{pmatrix} a \\ b \\ \mathbb{I}(a>b) \\ \mathbb{I}(b>a) \\ \vdots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \times \mathbb{I}(a>b) \\ b \times \mathbb{I}(b>a) \\ 0 \\ 0 \\ \vdots \end{pmatrix} = \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases}$$

Implementation of the special operation

The operation we are looking for to do it's not a linear transformation, therefore we need to define a new technique. Remember that one of $\mathbb{I}_a$ and $\mathbb{I}_b$ is a 1 and the other is a 0. Notice that $\mathbb{I}_b = 1 - \mathbb{I}_a$ and $\mathbb{I}_a = 1 - \mathbb{I}_b$.

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \times \mathbb{I}_a \\ b \times \mathbb{I}_b \\ 0 \\ 0 \end{pmatrix}$$

This operation can be performed with three layers of FF. What we will do is the following:

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a & + & \infty \times \mathbb{I}_b & + & \infty \times \mathbb{I}_b & + & -\infty \times \mathbb{I}_b \\ b & + & \infty \times \mathbb{I}_a & + & \infty \times \mathbb{I}_a & + & -\infty \times \mathbb{I}_a \\ \mathbb{I}_a & + & 0 & + & 0 & + & -\mathbb{I}_a \\ \mathbb{I}_b & + & 0 & + & 0 & + & -\mathbb{I}_b \end{pmatrix}$$

If $\mathbb{I}_a = 1$, then $\mathbb{I}_b = 0$, so the first coordinate isn't modified and the second is changed to 0 because of the finite representation properties. In the same way, if $\mathbb{I}_a = 0$, then $\mathbb{I}_b = 1$, so the second coordinate isn't modified and the first is changed to 0.

As mentioned before, this operation is performed with three FF layers. The first two will be:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & \infty \\ 0 & 0 & \infty & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Notice that the output of this FF layer when inputted with $(a, b, \mathbb{I}_a, \mathbb{I}_b)$ will be $(\infty \times \mathbb{I}_b, \infty \times \mathbb{I}_a, 0, 0)$.

The third FF layer will be:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ 0 & 0 & -\infty & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

which inputted with $(x, y, \mathbb{I}_a, \mathbb{I}_b)$, it will output $(-\infty \times \mathbb{I}_b, -\infty \times \mathbb{I}_a, -\mathbb{I}_a, -\mathbb{I}_b)$

Repeat a token once it's printed

There are going to be constructions were we would like to be able to, once the transformer prints a token, repeat it indefinitely. We can achieve this easily, making use of some of the tools that we had defined. First, we need to normalize the transformer. i know i define normalization later on, but i think this is the place in the text where i should write this primitive. Second, we need to flag the token with the TE. Third, we will apply a constant function to the flagged vector. Since we are trying to replicate a token that was printed, i.e. not part of the input, its vector will be the last one. This is because once the transformer prints it for the first time, we will start forcing it to repeat it. It's important that the constant function is applied in the last layers so that the vector that goes into the OUTPUT layer (which is just a projection) is the one making all the probability going to the desired token.

Notice that this only works if the token does not appear in the input word. That case can be solved using the non-uniformity of the model. If the input word is of size n, with the PE we can flag the first n vectors. Then, we can use that flag to make them 0 in the coordinate used by the TE for flagging the token to repeat. We make them 0 using a constant function as described in a previous section.

Notice that this primitive makes the assumption that a transformer uses all of its budget before halting not a problem. This is because if it printed a stopping symbol, but it still had budget, we can force it to repeat it until the end.

Force to halt after certain number of steps of CoT

With a similar strategy to the previous section, with the PE we can flag when a certain position is reached. Then, if the transformer is normalized, we can make the last vector what ever we want (since it will be the flagged by the PE). Particularly, we can force it to concenter the probability in q_{accept} or q_{reject}.

Make a vector ignore others in an ATTN layer

We will find very useful to be able for some vectors to ignore others in the ATTN layers. For a vector to ignore other, we will need for the attention score to be $-\infty$ so the softmax make it zero.

Let's say we want for the lth vector to ignore a set of vectors M. Therefore, what we are looking for is for $\langle q_l, k_m \rangle = -\infty$ when $m \in M$, but we don't want to modify the attention score of others pairs. We will use the saturation error for getting to our objective, since $\langle q_l, k_m \rangle = \sum_{x=1}^{d} (q_l)_x (k_m)_x$ if we make for the last two terms to be $-\infty$

when $m \in M$, we will get what we are looking for. Since we don't want to affect other vectors, every other pair should have 0 as the last to terms.

We will make $(k_i)_{d-1}$ and $(k_i)_d$ to be $-\infty$ when $i \in M$, and $(q_i)_{d-1}$ and $(q_i)_d$ to be ∞ when $i = l$ and 0 otherwise. With these values the only attention scores modified are the ones of the lth vector since every other will have 0 in the last two coordinates of the query thus making the last two terms of the internal product 0 and therefore not affecting the score.

For getting those values in the keys and queries vectors we can just flag them and adjust the query and key matrix to have what we want in those coordinates.

3.2 Normalization

For an easier handling of transformers, we define a notion of normalized transformer. This notion will specify properties of the OUTPUT matrix and of the vector consumed by this step.

Definition 3.2.1 (Normalized transformer)**.** We will say a transformer is normalized when it meets two conditions. First, the matrix of the OUTPUT has to be a projection, i.e. it's a diagonal matrix with only ones in its diagonal. Second, the last vector after the last iteration has to only be composed of $-\infty$ in every projected coordinate except, one which has to be ∞. The non projected coordinates are free to be any value.

The motivation for this definition is to simplify the OUTPUT layer (first constraint) and to have more control over the probability distribution generated (second constraint). The exact values of the last vectors are such that after projecting and applying softmax in the OUTPUT layer we get a deterministic distribution with 100% of the probability concentered in only one token.

Algorithm for normalizing

not sure about title

Let's see how to normalize any transformer. Let's take a transformer and modify it to meet the first constraint.

Every linear transformation $M : \mathbb{R}^d \to \mathbb{R}^{|\Sigma|}$ with $d \geq |\Sigma|$ can be decomposed in two steps. [1]

$$\begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \xrightarrow{M'} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \\ v \end{pmatrix} \xrightarrow{projection} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \end{pmatrix}$$

where $v \in \mathbb{R}^{d-|\Sigma|}$ can have any values.

Let's TF be a transformer and M be the matrix of the OUTPUT layer. Using the decomposition just presented, and the strategy defined in the previous section for applying linear transformations, we can change the matrix of the OUTPUT layer to be only a projection. The linear transformation has to be applied to the last vector.

[1] We can always add more coordinates to the embedding if $d < |\Sigma|$. Since the language is fixed, it will be a constant number of coordinates, thus not changing the class of the transformer.

Let's now make it meet the second condition.

Once TF has been modified to meet the first condition, we can use the maximum operation described in the previous section to make all the positions of the vector 0 except the one that was the maximum before. Now, if the maximum is positive, multiplying by ∞, then subtracting 1 and multiplying by ∞ again gets us what we wanted.

$$\begin{pmatrix} 0 \\ \vdots \\ 0 \\ x_{max} \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \infty \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{substracting 1}} \begin{pmatrix} -1 \\ \vdots \\ -1 \\ \infty - 1 \\ -1 \\ \vdots \\ -1 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} -\infty \\ \vdots \\ -\infty \\ \infty \\ -\infty \\ \vdots \\ -\infty \end{pmatrix}$$

It could be the case that the maximum was negative, in that case the resulting vector won't be what we are looking for. Let's see how to make the maximum strictly positive so we can add this mechanism in the middle.

By this point our vector looks like $(0, \dots, 0, x_{max}, 0, \dots, 0)$, we are going to transform it to $(0, \dots, 0, |x_{max}|, 0, \dots, 0)$. Remember that the coordinates we are interested in are only the first $|\Sigma|$. Let's call v to the block containing these coordinates. Our algorithm will do the following:

$$\begin{pmatrix} v \\ \vdots \end{pmatrix} \to \begin{pmatrix} v \\ -v \\ \vdots \end{pmatrix} \to \begin{pmatrix} \text{relu}(v) \\ \text{relu}(-v) \\ \vdots \end{pmatrix} \to \begin{pmatrix} \text{relu}(v) + \text{relu}(-v) \\ \vdots \end{pmatrix}$$

where relu is applied position wise. This works since for all x it holds that $\text{relu}(x) + \text{relu}(-x) = |x|$.

The first step can be done in the same way as the first half of the linear transformations. For the second step we can take max coordinate in v and $-v$. This works because all the coordinates are 0 except one, which if it is negative will become 0 and if it is non-negative will remain the same. For adding both vectors we can use one FF with the following parameters. Notice that this works since all the coordinates of $\text{relu}(-v)$ are non-negative.

$$W_1 = id^{d \times d} \qquad W_2 = \begin{pmatrix} 0^{|\Sigma| \times |\Sigma|} & id^{|\Sigma| \times |\Sigma|} & 0^{|\Sigma| \times (d-2*|\Sigma|)} \\ 0^{d-|\Sigma| \times |\Sigma|} & 0^{d-|\Sigma| \times |\Sigma|} & 0^{d-|\Sigma| \times d-2*|\Sigma|} \end{pmatrix} \qquad b_1 = b_2 = \left(0^{d \times d}\right)$$

3.3 Boolean Operations

The main objective of this chapter was to prove that the $\text{CoT}[T(n), d(n)]$ classes are closed under boolean operations.

Let's show that the complement of a language in $\text{CoT}[T(n), d(n)]$ is in $\text{CoT}[T(n), d(n)]$ and that the disjunction of two language of the same class, stays in the same class.

Complement of a language

Let's $\mathcal{L} \in \mathrm{CoT}[T(n), d(n)]$, then there is a family of transformers $\{\mathrm{TF}_n\}_{n\in\mathbb{N}}$ of embedding size $d(n)$ that after $T(n)$ steps of CoT prints q_{accept} or q_{reject} depending on if the input word is in the language. For each of the transformers in the family we can define one that behaves exactly the same (internally and externally) but prints the flipped token.

Let's TF be a transformer, we will construct TF' that flips the halting token of TF, i.e if $\mathrm{TF}^*(\alpha) = q_{accept}$, then $\mathrm{TF}'^*(\alpha) = q_{reject}$ and if $\mathrm{TF}^*(\alpha) = q_{reject}$, then $\mathrm{TF}'^*(\alpha) = q_{accept}$.

This can be easily done swapping the rows associated to q_{accept} and q_{reject} in the matrix of the OUTPUT layer of TF. Thus, when the largest probability is associated to q_{accept} in TF, that same number will be associated to q_{reject} in TF.

Formally, TF' will be exactly the same as TF except in the OUTPUT layer, where the matrix will be different. If M is the matrix of the OUTPUT layer of TF, i is the position of the token q_{accept} and j the one of q_{reject}, then the matrix of the OUTPUT layer of TF will be $M' = PM$ where P is the matrix that flips the rows i and j.

$$M'_{kl} = \begin{cases} 1 & \text{if } k = l,\ k \neq i \text{ and } k \neq j \\ 1 & \text{if } k = j \text{ and } l = i \\ 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

TF' has the same budget and embedding size that TF.

We can define the family $\{\mathrm{TF}'_n\}_{n\in\mathbb{N}}$ where TF'_i is the negation of TF_i. Then $\{\mathrm{TF}'_n\}_{n\in\mathbb{N}}$ computes the complement of $\{\mathrm{TF}_n\}_{n\in\mathbb{N}}$. Since for every $i \in \mathbb{N}$, TF'_i has the same budget and embedding size that TF_i, $\bar{\mathcal{L}}$ is in $\mathrm{CoT}[T(n), d(n)]$.

Disjunction of two languages

4. MAIN RESULTS

5. CONCLUSIONS

BIBLIOGRAPHY

[1] A. Bick, A. Blandin, and D. J. Deming. The rapid adoption of generative AI. Management Science, 1 2026.

[2] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of thought empowers transformers to solve inherently serial problems. In The Twelfth International Conference on Learning Representations, 2024.

[3] W. Liang, Y. Zhang, Z. Wu, H. Lepp, W. Ji, X. Zhao, H. Cao, S. Liu, S. He, Z. Huang, D. Yang, C. Potts, C. D. Manning, and J. Y. Zou. Mapping the increasing use of llms in scientific papers, 2024.

[4] T. Lin, Y. Wang, X. Liu, and X. Qiu. A survey of transformers. AI Open, 3:111–132, 2022.

[5] A. Madaan and A. Yazdanbakhsh. Text and patterns: For effective chain of thought, it takes two to tango. arXiv preprint arXiv:2209.07686, 2002.

[6] W. Merrill and A. Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers. arXiv preprint arXiv:2503.03961, 2025.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 30. Curran Associates, Inc., 2017.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[9] B. Wang, S. Min, X. Deng, J. Shen, Y. Wu, L. Zettlemoyer, and H. Sun. Towards understanding chain-of-thought prompting: An empirical study of what matters. In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 2717–2739, Toronto, Canada, July 2023. Association for Computational Linguistics.

[10] A. Yang, C. Watson, A. Xue, S. Bhattamishra, J. Llarena, W. Merrill, E. D. S. Ferreira, A. Svete, and D. Chiang. The transformer cookbook, 2025.